Table 8: Dataset Statistics. Note that the Math datasets (MultiArith and GSM8K) are used for few-shot prompting with GPT-J and GPT-3.

Dataset	# train	# valid	# test	In domain	Out-of domain
Toy-story	10k	1k	1k	1, 2 hops	3, 4 hops
Algorithm	10k	1k	1k	2-100 statements	101-200 statements
Boolean Variable	524k	1k	1k	3-8 statements	9-19 statements
Chess Piece	200k	1k	1k	$\leq$ 80 moves	$\geq$ 81 moves
Chess Move	200k	1k	1k	$\leq$ 80 moves	$\geq$ 81 moves
MultiArith	8 (prompt)	N/A	600	N/A	N/A
GSM8K	5 (prompt)	N/A	1319	N/A	N/A

GSM8K. This task was introduced to evaluate more challenging arithmetic reasoning problems [19]. We selected 5 random samples in the training set to use as the few-shot demonstrations in the prompt. For Chain-of-Thought, we used the original solutions (minus the calculation annotations written inside "« »") given in the dataset. To create the Self-Note prompts, we remove the Chain-of-Thought explanations in the answer and add our own Self-Note enrichments in the 5 prompts. The start and end tokens are the same as MultiArith: "`(`"and "`)`". Table 13 shows the exact prompts used for the Self-Notes and Chain-of-Thought results. Self-Notes are only added after ".", "," and "?" tokens, so we do not have to call the API after each token.

A.2 Training Details

For each non-prompting task, we train for a fixed 30 epochs with a learning rate of $2e$-5 and batch size of 32. For the Vanilla and Self-Notes methods, all models are trained on the context and answer tokens. For the Scratchpad method, the model is trained on the context and answer for Toy-Story, and is trained on only the answer for all other tasks since the scratchpad contains a full copy of the context. For the tasks that involve length generalization (Algorithmic, Boolean Variable, Chess Piecetype, Chess Move) we randomly offset the positional embedding IDs between 0 and 128 for the GPT-2 model during training. That is, the positional embedding ID of the first token is randomly assigned to a value from 0 to 128, and then continues sequentially. During inference, the position IDs always start at 0. This ensures that the model has seen all of the position embeddings that it will encounter during testing (following Kiyono et al. [38]). We note that this can also be solved by the relative position embeddings used in more recent models [49, 50].

We fine-tune all of the GPT-2 models on 8 NVIDIA V100 GPUs using an on-site cluster. The GSM8K experiments were done using the text-davinci-003 model with the OpenAI API, costing around $300.

A.3 Dummy Tokens: Separating Content and Compute

We introduced Self-Notes as a method for language models to do reasoning in the form of explicit tokens. To understand the value of these extra tokens, we seek to separate and measure the contribution of the additional compute that is allotted by these tokens from their content. In other words, we answer the question "Is it the specific tokens in the Self-Notes or is it simply extra steps of computation that is helping?". We do this by inserting the dummy token "`_`" at various locations throughout the context as an ablation. These are similar to a Self-Note with a single token that always remains the same.

We first consider the Toy-Story task. In the vanilla setting for this task, there are only facts and the question (e.g., "`Bob is in the park. Bob has the key. Where is the key?`"). The first comparison is inserting a dummy after every fact, which we call "Naïve Dummy" ("`Bob is in the park. _ Bob has the key. _ Where is the key?`"). The alternative, which we call "Self-Notes Dummy" is adding dummy tokens in the same locations where Self-Notes are written. In other words, at the locations where a relation between two facts can be inferred ("`Bob is in the park. Bob has the key. _ Where is the key?`"). Finally, we consider adding the dummy tokens where the Scratchpad tokens would be ("`Bob is in the park. Bob has the key. Where is the key? _`*"). This setting adds the same amount of dummy tokens as are used in Self-Notes Dummy (only 1 in the example described, but the "$*$" operator indicates that extra dummy tokens can be added). Figure 6 shows the results on the 3-hop and 4-hop test sets for the different dummy token settings compared to the natural language tokens used by Self-Notes and Scratchpad. Intelligently inserting dummy tokens into the

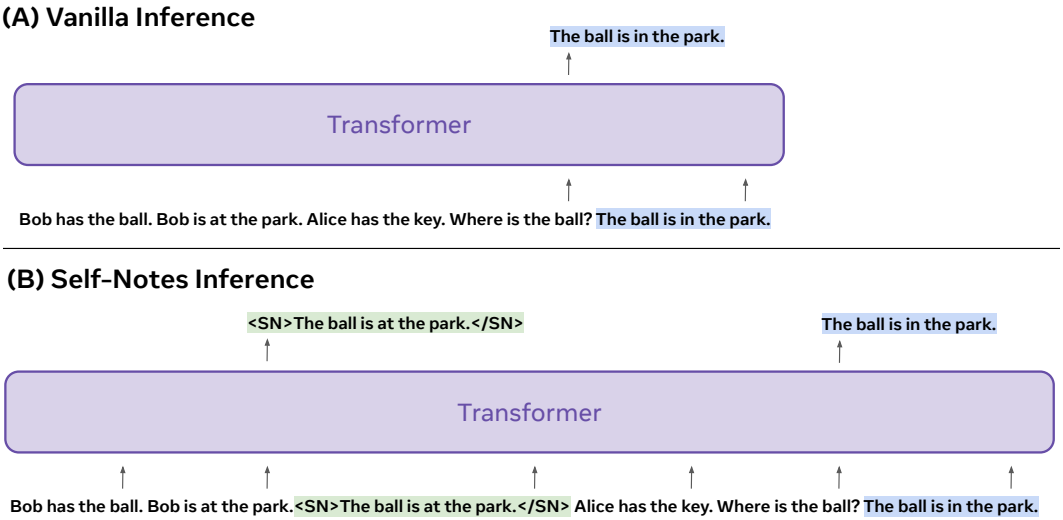

Figure 5: Comparison of Vanilla vs Self-Notes inference. Non-highlighted tokens are given to the model. Highlighted tokens are generated by the model. In Vanilla inference (top), the model generates only after the full context and question are provided. In Self-Notes inference (bottom), the model is able to generate after every word or sentence in the context. If the next most likely token is the Self-Notes start token "<SN>", then the model can autoregressively generate itself a note until the end token "</SN>" is generated, at which point it returns to reading the original context.

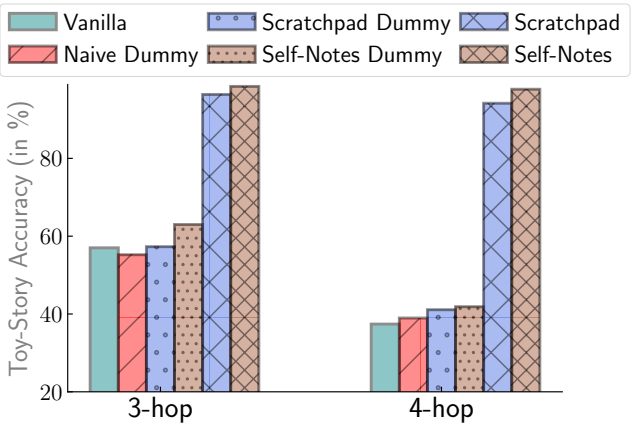

Figure 6: Ablation comparing the impact of (a) extra compute due to additional tokens, and (b) position of the additional tokens.

positions where Self-Notes should be performs the best out of the four settings. Importantly, it is better than inserting at the end of the context, where Scratchpad tokens are added. This alludes to the fact that *allowing the model to do extra computations in the middle of the context can be better than after the context*.[5] However, the gain from Self-Notes Dummy over other dummy variants pales in comparison to the gains of actual Self-Notes over its dummy counterpart, suggesting that the content of the intermediate notes matters more than just the additional compute.

We also test the usefulness of Self-Notes Dummy tokens in three other settings: Chess Piecetype, Chess Move, and WikiText-103 language modeling. For the chess experiments, we compare the vanilla moves ("`c2 c4`"), a dummy token between move positions ("`c2 _ c4`"), and the generated

[5]In the in-context learning setting, Wei et al. [4] reported that gains with chain-of-thought were not due to additional compute. Our setting though differs in three aspects: (a) the additional compute is happening in the middle of the context in case of Self-Notes Dummy, (b) we're finetuning the language model, and (c) we're working with a much smaller language model.

(non-dummy) piece tokens in Self-Notes ("`c2 P c4`"). For WikiText-103 language modeling, we compare the vanilla text ("`The cat sat on the mat`") with dummy tokens inserted between each word in the text ("`The _ cat _ sat _ on _ the _ mat`"). There are no ground-truth Self-Notes for WikiText-103, so we neglect such experiments. Table 10 shows the accuracy results for chess and perplexity for WikiText-103. Dummy tokens are not reported in the accuracy or perplexity numbers. For each task, dummy tokens improves the vanilla setting by a small margin.

A.4 Further Ablations

Oracle/No Self-Notes during inference. We perform two ablations regarding the use of Self-Notes during inference. The first is an upper bound where we provide 100% Self-Notes supervision to the model during both training and inference (rather than just training). The second is where we give 100% Self-Notes supervision during training, but restrict the model from generating Self-Notes during inference. This baseline analyzes whether Self-Notes-augmented training data can still help the model learn about the task in its weights even without using Self-Notes at inference time. The results are shown in Table 11. As expected, oracle Self-Notes (100% Self-Notes supervision during inference) improves the performance. On the other hand, not allowing the model to generate Self-Notes leads to a drastic drop in performance due to distribution shift at inference time.

A.5 Prompts for Few-Shot Self-Notes

For the results in Table 4 and Table 5, we use the Self-Notes prompts shown in Table 12 and Table 13, respectively. The MultiArith prompt (Table 12) is the same set of examples as used in Wei et al. [4]. We augment the standard prompts they use with Self-Notes, which are interleaved in the context and question. We use a GSM8K prompt (Table 13) derived from a subset of the training samples, where we added Self-Notes. All prompting experiments are compared to Vanilla (standard prompts) and Chain-of-thought (COT) prompts.